TRABALHO DE AVALIAÇÃO — SIMULAÇÃO DE UM MULTIBANCO

1. Objetivo

Desenvolva, em Python, um programa que simule o funcionamento de um multibanco com dois perfis de utilização: cliente e administrador. O cliente consulta o saldo, faz levantamentos, depósitos e transferências e consulta os últimos movimentos. O administrador cria, lista, elimina e desbloqueia contas.

O trabalho avalia os conteúdos dos Módulos 2 a 4: estruturas de controlo, estruturas de dados, expressões regulares e biblioteca `re`, funções, exceções e Programação Orientada a Objetos (encapsulamento, herança, polimorfismo e composição).

2. Regras do Trabalho

- O trabalho é individual.
- São dedicadas ao trabalho três aulas de 4 horas. O trabalho pode ser continuado fora das aulas, para terminar e rever o que ficou por fazer.
- Prazo de entrega: 21/10/2026, até às 23h59.
- Entrega: um ficheiro ZIP com o nome `multibanco_nome_apelido.zip` (por exemplo, `multibanco_luis_vigario.zip`), que contém a pasta `multibanco` com os 9 ficheiros indicados na secção 3.
- Versão do Python: 3.10 ou superior. Só pode ser usada a biblioteca padrão do Python.
- Regras de escrita do código: nomes de variáveis, funções, classes e métodos em inglês (snake_case e PascalCase); type hints em todas as funções e métodos; docstrings e comentários em português; mensagens ao utilizador em português. Não é permitido usar `global` nem `except` genérico.
- O trabalho parte do esqueleto fornecido pelo formador (secção 3). Não podem ser alterados os nomes, os parâmetros, os type hints nem as docstrings dos métodos e funções fornecidos, nem as constantes de classe, nem os ficheiros `data.py` e `messages.py`. Podem ser acrescentados métodos e funções auxiliares.
- As mensagens apresentadas ao utilizador têm de ser exatamente as indicadas na secção 7, porque a avaliação é feita com os cenários de teste da secção 9. As mensagens fixas são usadas através das constantes de `messages.py` (por exemplo, `messages.INSUFFICIENT_FUNDS`) e nunca reescritas no código.
- É fornecido também o script `corrigir_cenarios.py`, que corre automaticamente os cenários da secção 9. É o mesmo script usado na avaliação.

Penalizações

- Entrega fora do prazo: menos 1 valores por cada dia de atraso.
- Se o programa não arrancar (o comando `python main.py` termina com erro), os critérios que dependem da execução são avaliados apenas pela leitura do código, com um máximo de 0,8 em cada um.
- Trabalhos com código substancialmente igual entre alunos são anulados.

3. Estrutura do Projeto

Cada classe fica no seu próprio ficheiro. O programa é executado com `python main.py`, a partir da pasta `multibanco`.

O esqueleto fornecido já contém os 9 ficheiros. Em cada método e função a implementar estão escritos a assinatura e a docstring, e o corpo tem só as linhas `# TODO: implementar` e `raise NotImplementedError`, que devem ser substituídas pelo código. As docstrings indicam, na secção `Raises`, a constante de `messages.py` a usar em cada erro.

Ficheiro	Conteúdo
`movement.py`	Classe `Movement`.
`account.py`	Classe `Account` (classe base das contas).
`checking_account.py`	Classe `CheckingAccount` (conta à ordem).
`savings_account.py`	Classe `SavingsAccount` (conta poupança).
`bank.py`	Classe `Bank`.
`validators.py`	Funções de validação e de conversão de dados. Não contém classes.
`messages.py`	Mensagens fixas e designações (tipos de conta e de movimento), em constantes. Fornecido completo; não pode ser alterado.
`data.py`	Dados iniciais (secção 8). Fornecido completo; não pode ser alterado.
`main.py`	Menus e interação com o utilizador. Fornecido só com os `import`, a constante `ADMIN_PIN = "9999"` e o início da função `main()`; o resto é escrito pelo formando.

4. Regras de Funcionamento

Esta secção define todas as regras do programa. Em caso de dúvida durante o trabalho, é esta secção que decide.

4.1. Dados das contas

- Número de conta: exatamente 4 dígitos.
- Titular: pelo menos nome e apelido, só com letras (os acentos são permitidos) e com um único espaço entre as palavras. Não são aceites números, hífenes nem apóstrofos (por exemplo, `Ana Lopes` e `José Fonseca` são válidos; `Ana`, `Ana-Maria Lopes` e `Ana  Lopes`, com dois espaços, não são).
- PIN: exatamente 4 dígitos.
- IBAN: `PT50` seguido de exatamente 21 dígitos (25 caracteres no total).
- Conta à ordem e conta poupança são os dois únicos tipos de conta. A classe `Account` reúne o que é comum às duas, mas o programa nunca cria objetos diretamente a partir dela.

4.2. Valores e saldo

- O utilizador pode usar o ponto ou a vírgula como separador decimal (`25.50` ou `25,50`).
- Em depósitos, levantamentos e transferências, o valor tem de ser positivo (maior que 0) e ter no máximo 2 casas decimais. Um valor negativo ou zero é recusado com a mensagem de valor inválido da secção 7, e não com a mensagem de texto inválido.
- Depósito inicial de uma conta nova: pode ser 0, mas não pode ser negativo nem ter mais de 2 casas decimais. Não regista nenhum movimento.
- Não existe valor máximo para depósitos nem para transferências.
- Depois de cada operação, o saldo é arredondado a 2 casas decimais com `round(saldo, 2)`. Sem este arredondamento, contas como 0,10 + 0,20 deixam resíduos (0,30000000000000004) que, mais tarde, impedem, por exemplo, eliminar uma conta com saldo zero.
- Os valores são sempre mostrados com 2 casas decimais e ponto, seguidos de `€` (por exemplo, `1130.75 €` ou `-150.00 €`).
- Consultar saldo mostra o saldo real da conta, que numa conta à ordem pode ser negativo. Não inclui o valor ainda disponível no descoberto.

4.3. Operações

- Levantamentos: o valor tem de ser múltiplo de 10 €. Os depósitos e as transferências não têm esta restrição (por exemplo, é possível transferir 25,50 €).
- Conta à ordem: os levantamentos e as transferências podem deixar o saldo negativo até -200,00 € (descoberto).
- Conta poupança: o saldo nunca pode ficar negativo. Cada levantamento cobra uma comissão de 1,00 €, registada como um movimento separado, a seguir ao levantamento. O saldo tem de cobrir o valor mais a comissão. As transferências não têm comissão.
- Transferências: só entre contas do banco, indicando o IBAN de destino. O IBAN introduzido é convertido para maiúsculas e sem espaços nas pontas (`pt50…` é aceite). Não é possível transferir para a própria conta. A transferência fica registada nas duas contas e, se falhar, nenhuma das contas é alterada. Uma conta bloqueada continua a poder receber transferências.
- Movimentos: cada conta numera os seus movimentos a partir de 1. O valor é positivo nos créditos (depósitos e transferências recebidas) e negativo nos débitos (levantamentos, transferências enviadas e comissões). A consulta mostra os 10 mais recentes, do mais recente para o mais antigo; se houver menos de 10, mostra todos.

4.4. Acesso e bloqueio

- Cliente: se a conta não existir, o programa mostra a mensagem e volta ao menu inicial. Se a conta estiver bloqueada, mostra a mensagem e volta ao menu inicial, sem pedir o PIN. Caso contrário, pede o PIN até estar certo ou até a conta ficar bloqueada.
- Qualquer PIN diferente do correto conta como falha, incluindo um PIN com letras ou com mais ou menos de 4 dígitos. À 3.ª falha seguida, a conta fica bloqueada e o programa volta ao menu inicial. Um PIN certo repõe a contagem a zero.
- Só o administrador pode desbloquear uma conta. O desbloqueio repõe também a contagem de falhas.
- Administrador: o PIN é `9999`, guardado numa constante no `main.py`. Cada entrada no menu de administrador tem uma tentativa: se o PIN estiver errado, o programa mostra a mensagem e volta ao menu inicial. Não há limite de tentativas nem bloqueio, porque o administrador não é uma conta do banco.

4.5. Administração

- Nova conta: o número é o maior número existente mais 1, e o IBAN é `PT50` seguido do número com 21 dígitos (com zeros à esquerda). Por exemplo, a conta 1004 tem o IBAN `PT50000000000000000001004`. Se a conta com o maior número for eliminada, esse número volta a ser usado pela conta seguinte.
- Eliminar conta: só é possível com saldo exatamente zero. Uma conta à ordem com saldo negativo também não pode ser eliminada. Uma conta bloqueada com saldo zero pode.
- Desbloquear conta: só é possível numa conta bloqueada.

4.6. O que não é pedido

Para evitar trabalho desnecessário, os pontos seguintes ficam fora do trabalho e não são avaliados:

- guardar os dados num ficheiro (ao sair, as alterações perdem-se e o programa volta a arrancar com os dados da secção 8);
- datas e horas dos movimentos;
- transferências para contas de outros bancos;
- alterar o PIN, o titular ou o tipo de uma conta existente;
- juros ou outras comissões além da comissão de levantamento da conta poupança;
- o limite de 9999 contas;

5. Classes e Funções Obrigatórias

As tabelas seguintes descrevem o comportamento dos elementos que já estão no esqueleto, com os nomes, os parâmetros e o tipo devolvido. Podem ser criados métodos ou funções adicionais, desde que privados ou auxiliares. Para facilitar a leitura, as mensagens aparecem aqui por extenso; no código, usa-se sempre a constante correspondente de `messages.py`.

Os métodos com um underscore (_) são protegidos: são usados pela própria classe e pelas classes derivadas, mas nunca pelo `main.py` nem pelo `Bank`. Existem porque as classes derivadas não conseguem aceder aos atributos privados da classe base (por exemplo, `__balance`): o name mangling impede-o.

5.1. validators.py

Funções sem classes. As quatro primeiras só verificam o formato e devolvem `True` ou `False`; não levantam exceções.

Método / atributo	Devolve	O que faz
`validate_account_number(number: str)`	`bool`	Exatamente 4 dígitos, com `re.fullmatch`.
`validate_pin(pin: str)`	`bool`	Exatamente 4 dígitos, com `re.fullmatch`.
`validate_iban(iban: str)`	`bool`	`PT50` seguido de exatamente 21 dígitos, com `re.fullmatch`.
`validate_name(name: str)`	`bool`	Nome e apelido, só com letras e um espaço entre palavras, com `re.fullmatch` (regras da secção 4.1).
`validate_amount(amount: float)`	`bool`	`True` se o valor for maior que 0 e tiver no máximo 2 casas decimais (`round(amount, 2) == amount`).
`parse_amount(text: str)`	`float`	Converte o texto num número, aceitando o sinal negativo, o ponto e a vírgula. Retira os espaços nas pontas. Se o texto não for um número (por exemplo, `abc` ou `12,5,0`), levanta `ValueError("Valor inválido. Introduza um número.")`. Não verifica se o valor é positivo: isso é feito por `validate_amount`.

5.2. Movement

Um movimento é criado pela conta e nunca é alterado depois (só tem properties de leitura).

Método / atributo	Devolve	O que faz
`__init__(self, number: int, movement_type: str, amount: float, description: str)`	`None`	Guarda os quatro dados em atributos privados. Não valida: os dados chegam já validados pela conta.
Properties `number`, `movement_type`, `amount`, `description`	—	Só de leitura.
`__str__(self)`	`str`	`#001 | Depósito | +200.00 € | Depósito em numerário`: número com 3 dígitos, valor sempre com sinal (`+` ou `-`) e 2 casas decimais.
`__repr__(self)`	`str`	`Movement(1, 'Depósito', 200.0, 'Depósito em numerário')`

5.3. Account (classe base)

Constantes de classe: `MAX_PIN_ATTEMPTS = 3` e `WITHDRAWAL_MULTIPLE = 10`. Todos os atributos de instância são privados: número, titular, PIN, IBAN, saldo, a lista de objetos `Movement` (composição), o contador de PIN errados e o estado de bloqueio.

O construtor valida os dados por esta ordem e, no primeiro dado inválido, levanta `ValueError` com a mensagem da secção 7: número, titular (através do setter de `holder`), PIN, IBAN e saldo inicial.

Método / atributo	Devolve	O que faz
`__init__(self, number: str, holder: str, pin: str, iban: str, initial_balance: float = 0.0)`	`None`	Valida e guarda os dados, com o contador de falhas a 0, a conta desbloqueada e a lista de movimentos vazia. O saldo inicial não regista nenhum movimento.
Properties `number`, `iban`, `balance`, `is_blocked`	—	Só de leitura: não têm setter. O saldo só muda através dos métodos de operação.
Property `holder`	—	Leitura e escrita. O setter valida o nome com `validate_name` e é usado pelo próprio construtor (`self.holder = holder`).

Método / atributo	Devolve	O que faz
`account_type(self)`	`str`	Devolve o nome do tipo de conta, para ser mostrado ao utilizador. Na classe base devolve `"Conta"`, um valor genérico que nunca chega a aparecer, porque o programa só cria contas à ordem e poupança. Cada classe derivada redefine este método. O `__str__` da classe base chama `self.account_type()` e o Python executa a versão da classe do objeto: o mesmo `__str__` mostra `Conta à ordem` ou `Conta poupança` sem nenhum `if` (polimorfismo). Por isso a listagem de contas não precisa de testar o tipo de cada conta.
`check_pin(self, pin: str)`	`bool`	Se a conta estiver bloqueada, levanta `RuntimeError("Conta bloqueada. Contacte o administrador.")`: é uma proteção da própria classe, porque o `main.py` já não deve pedir o PIN de uma conta bloqueada. Se o PIN estiver certo, repõe o contador a 0 e devolve `True`. Se estiver errado, soma 1 ao contador; quando chega a `MAX_PIN_ATTEMPTS`, bloqueia a conta. Devolve `False`.
`unblock(self)`	`None`	Desbloqueia a conta e repõe o contador a 0.
`deposit(self, amount: float)`	`None`	Valida o valor com `_validate_amount` e chama `_credit` com o tipo `Depósito` e a descrição `Depósito em numerário`.
`withdraw(self, amount: float)`	`None`	Valida o valor e o múltiplo de 10 com `_validate_withdrawal`. Se o valor for maior que `_available_funds()`, levanta `ValueError("Saldo insuficiente.")`. Depois chama `_debit` com o tipo `Levantamento` e a descrição `Levantamento ATM`.
`transfer_out(self, amount: float, destination_iban: str)`	`None`	Usado pelo `Bank`. Valida o valor com `_validate_amount` e os fundos com `_available_funds()` (sem múltiplo de 10), e chama `_debit` com o tipo `Transferência enviada` e a descrição `Para <IBAN de destino>`.
`transfer_in(self, amount: float, origin_iban: str)`	`None`	Usado pelo `Bank`. Valida o valor e chama `_credit` com o tipo `Transferência recebida` e a descrição `De <IBAN de origem>`.

Método / atributo	Devolve	O que faz
`get_last_movements(self, count: int = 10)`	`list[Movement]`	Devolve os últimos `count` movimentos, do mais recente para o mais antigo. Se houver menos, devolve todos; se não houver nenhum, devolve uma lista vazia. Não altera a lista interna.
`_available_funds(self)`	`float`	Valor máximo que pode sair da conta. Na classe base devolve o saldo. `withdraw` e `transfer_out` comparam o valor com este método, e não diretamente com o saldo: assim, a conta à ordem só tem de o redefinir para permitir o descoberto, sem repetir `withdraw` nem `transfer_out`.
`_validate_amount(self, amount: float)`	`None`	Levanta `ValueError("O valor tem de ser positivo e ter no máximo 2 casas decimais.")` se `validate_amount` devolver `False`.
`_validate_withdrawal(self, amount: float)`	`None`	Chama `_validate_amount` e, se o valor não for múltiplo de `WITHDRAWAL_MULTIPLE`, levanta `ValueError("O valor do levantamento tem de ser múltiplo de 10 €.")`. Não verifica os fundos. É usado por `withdraw` nas duas classes, para a ordem das validações ser sempre a mesma.
`_credit(self, amount: float, movement_type: str, description: str)`	`None`	Soma o valor ao saldo, arredonda-o a 2 casas decimais e cria o `Movement` seguinte, com valor positivo. Não valida: quem o chama já validou.
`_debit(self, amount: float, movement_type: str, description: str)`	`None`	Subtrai o valor ao saldo, arredonda-o a 2 casas decimais e cria o `Movement` seguinte, com valor negativo. Recebe o valor positivo. É o método que a conta poupança usa para registar a comissão.
`__str__(self)`	`str`	`1001 | Conta à ordem | João Silva | 1130.75 €`, seguido de `| BLOQUEADA` se a conta estiver bloqueada. Usa `self.account_type()`.
`__repr__(self)`	`str`	`CheckingAccount('1001', 'João Silva', 1130.75)`. Usa `type(self).__name__`, para mostrar o nome da classe derivada sem ter de redefinir o método.

5.4. CheckingAccount e SavingsAccount

As duas classes herdam de `Account` e não definem `__init__` próprio: usam o construtor da classe base. Só redefinem os métodos indicados.

`CheckingAccount` (conta à ordem):

Método / atributo	Devolve	O que faz
`OVERDRAFT_LIMIT = 200.0`	—	Constante de classe: limite do descoberto.
`account_type(self)`	`str`	Devolve `"Conta à ordem"`.
`_available_funds(self)`	`float`	Devolve o saldo mais `OVERDRAFT_LIMIT`. Com saldo de -150 €, ainda podem sair 50 €.

`SavingsAccount` (conta poupança). Não redefine `_available_funds`: os fundos são o saldo, por isso o saldo nunca fica negativo.

Método / atributo	Devolve	O que faz
`WITHDRAWAL_FEE = 1.0`	—	Constante de classe: comissão de cada levantamento.
`account_type(self)`	`str`	Devolve `"Conta poupança"`.
`withdraw(self, amount: float)`	`None`	Estende o levantamento da classe base, por esta ordem: (1) `_validate_withdrawal(amount)`; (2) se `amount + WITHDRAWAL_FEE` for maior que `_available_funds()`, levanta `ValueError("Saldo insuficiente para o levantamento e a comissão.")`; (3) `super().withdraw(amount)`, que regista o levantamento; (4) `_debit` da comissão, com o tipo `Comissão` e a descrição `Comissão de levantamento`. As validações repetidas dentro de `super().withdraw` não falham, porque já foram feitas.

5.5. Bank

O banco guarda as contas num dicionário privado, em que a chave é o número da conta e o valor é o objeto da conta (composição). Os métodos que recebem um número de conta não validam o formato: um número inválido é simplesmente uma conta que não existe.

Método / atributo	Devolve	O que faz
`__init__(self, name: str)`	`None`	Guarda o nome e cria o dicionário de contas vazio.
`add_account(self, account: Account)`	`None`	Se o número já existir, levanta `ValueError("Já existe uma conta com o número <número>.")`; se o IBAN já existir, `ValueError("Já existe uma conta com o IBAN <IBAN>.")`.
`find_account(self, number: str)`	`Account | None`	Conta com esse número, ou `None`.
`find_by_iban(self, iban: str)`	`Account | None`	Conta com esse IBAN, ou `None`.
`list_accounts(self)`	`list[Account]`	Todas as contas, ordenadas pelo número.
`next_account_number(self)`	`str`	Maior número existente mais 1, com 4 dígitos (`"1004"` com os dados iniciais).
`generate_iban(self, number: str)`	`str`	`PT50` seguido do número com 21 dígitos: `f"PT50{int(number):021d}"`.
`remove_account(self, number: str)`	`None`	Se a conta não existir, `ValueError("Conta não encontrada.")`; se o saldo não for exatamente 0, `ValueError("Só é possível eliminar contas com saldo zero.")`.
`unblock_account(self, number: str)`	`None`	Se a conta não existir, `ValueError("Conta não encontrada.")`; se não estiver bloqueada, `ValueError("A conta não está bloqueada.")`. Caso contrário, chama `unblock()` da conta.
`transfer(self, origin: Account, destination_iban: str, amount: float)`	`None`	Valida por esta ordem: (1) formato do IBAN, com `validate_iban` (`IBAN inválido.`); (2) a conta de destino existe (`IBAN de destino não encontrado.`); (3) não é a conta de origem (`Não é possível transferir para a própria conta.`). Depois chama `origin.transfer_out(...)`, que valida o valor e os fundos, e só então `destination.transfer_in(...)`. Se `transfer_out` falhar, nenhuma conta é alterada.

5.6. main.py

O `main.py` cria o banco (`Bank("Banco ATEC")`), carrega os dados iniciais com `load_initial_data` e apresenta os menus. Contém só a interação com o utilizador: pede os dados, chama os métodos das classes e mostra as mensagens. As regras de negócio (validações, saldos, bloqueio) ficam nas classes.

O esqueleto só traz o início de `main()`. A organização em funções é da responsabilidade do formando e é avaliada; uma possibilidade é usar `read_amount`, `login`, `client_session`, `create_account`, `admin_session` e `main`, todas com type hints e docstring. Todos os `ValueError` levantados pelas classes são apanhados nos menus e mostrados com o prefixo `messages.ERROR_PREFIX` (`Erro: `), sem o programa terminar. A execução é protegida por `if __name__ == "__main__":`, que já está no esqueleto.

6. Menus

Os três menus funcionam em ciclo até o utilizador escolher a última opção. Uma opção inexistente, incluindo letras ou uma opção vazia, mostra `Opção inválida.` e o menu volta a aparecer. Terminar a sessão de cliente ou de administrador volta ao menu inicial; só a opção Sair do menu inicial termina o programa.

Menu inicial:

```
=== Multibanco ===
1. Entrar como cliente
2. Entrar como administrador
3. Sair
```

Menu de cliente:

```
1. Consultar saldo
2. Levantamento
3. Depósito
4. Transferência
5. Últimos 10 movimentos
6. Terminar sessão
```

Menu de administrador:

```
1. Criar conta
2. Listar contas
3. Eliminar conta
4. Desbloquear conta
5. Terminar sessão
```

6.1. Ordem dos pedidos e das validações

O texto de cada pedido é livre, mas a ordem dos dados pedidos tem de ser esta, porque os cenários de teste dependem dela. Em todas as operações, os erros mostram a mensagem e o programa volta ao mesmo menu.

- Entrar como cliente: (1) número da conta; (2) se não existir ou estiver bloqueada, mensagem e volta ao menu inicial; (3) PIN, pedido de novo após cada falha, até estar certo ou a conta ficar bloqueada.
- Entrar como administrador: PIN de administrador, uma vez; se estiver errado, mensagem e volta ao menu inicial.
- Levantamento e depósito: valor.
- Transferência: IBAN de destino e, depois, valor. Os dois dados são pedidos antes de qualquer validação; se o valor não for um número, a mensagem é a de valor inválido, mesmo que o IBAN também esteja errado.
- Criar conta: (1) tipo (`1` para à ordem, `2` para poupança); se o tipo for inválido, mostra `Erro: Tipo de conta inválido.` e não pede mais nada; (2) nome do titular, PIN e depósito inicial, pedidos todos antes de a conta ser criada. O construtor valida-os pela ordem da secção 5.3 e o primeiro erro é mostrado.
- Eliminar e desbloquear conta: número da conta.

7. Mensagens Obrigatórias

As mensagens têm de ser exatamente as indicadas, incluindo a pontuação. As mensagens de erro dos menus de cliente e de administrador são as das exceções `ValueError` levantadas pelas classes e por `parse_amount`, apresentadas com o prefixo `messages.ERROR_PREFIX` (por exemplo, `Erro: Saldo insuficiente.`). As mensagens de login e do PIN de administrador são mostradas sem prefixo.

7.1. Mensagens fixas

O texto destas mensagens está no ficheiro `messages.py`. A tabela indica a constante a usar em cada situação.

Situação	Constante em messages.py
Conta inexistente (login, eliminar, desbloquear)	`ACCOUNT_NOT_FOUND`
PIN errado (1.ª e 2.ª falha)	`WRONG_PIN`
3.ª falha seguida	`ACCOUNT_NOW_BLOCKED`
Entrar numa conta bloqueada	`ACCOUNT_BLOCKED`
PIN de administrador errado	`WRONG_ADMIN_PIN`
Conta sem movimentos	`NO_MOVEMENTS`
Terminar sessão (cliente e administrador)	`SESSION_ENDED`
Texto que não é um número	`INVALID_NUMBER_TEXT`

Situação	Constante em messages.py
Valor zero, negativo ou com mais de 2 casas decimais	`INVALID_AMOUNT`
Levantamento que não é múltiplo de 10	`INVALID_WITHDRAWAL_MULTIPLE`
Fundos insuficientes (levantamento ou transferência)	`INSUFFICIENT_FUNDS`
Poupança sem fundos para o valor e a comissão	`INSUFFICIENT_FUNDS_WITH_FEE`
IBAN com formato inválido (transferência ou criação de conta)	`INVALID_IBAN`
IBAN de destino inexistente	`DESTINATION_NOT_FOUND`
Transferência para a própria conta	`SAME_ACCOUNT_TRANSFER`
Número de conta com formato inválido	`INVALID_ACCOUNT_NUMBER`
Nome do titular inválido	`INVALID_HOLDER`
PIN com formato inválido (criação de conta)	`INVALID_PIN`
Saldo inicial negativo ou com mais de 2 casas decimais	`INVALID_INITIAL_BALANCE`
Tipo de conta inválido	`INVALID_ACCOUNT_TYPE`
Eliminar conta com saldo diferente de zero	`ACCOUNT_HAS_BALANCE`
Desbloquear conta não bloqueada	`ACCOUNT_NOT_BLOCKED`
Conta eliminada / desbloqueada	`ACCOUNT_REMOVED` e `ACCOUNT_UNBLOCKED`
Opção inexistente num menu	`INVALID_OPTION`
Sair do programa	`GOODBYE`
Prefixo das mensagens de erro nos menus	`ERROR_PREFIX`
Designação dos tipos de conta (`account_type`)	`ACCOUNT_TYPE_BASE`, `ACCOUNT_TYPE_CHECKING` e `ACCOUNT_TYPE_SAVINGS`
Tipos e descrições dos movimentos	Constantes começadas por `MOVEMENT_` (por exemplo, `MOVEMENT_DEPOSIT` e `MOVEMENT_DEPOSIT_DESCRIPTION`)

7.2. Mensagens com valores

As mensagens seguintes incluem valores e não estão em `messages.py`: são escritas no código com f-strings. As partes entre < e > são substituídas pelos valores; os saldos são mostrados com 2 casas decimais (`{balance:.2f}`).

Situação	Mensagem
Login com sucesso	`Bem-vindo(a), <titular>.`
Consultar saldo	`Saldo disponível: <saldo> €`
Levantamento, depósito e transferência com sucesso	`Levantamento efetuado. Saldo: <saldo> €`, `Depósito efetuado. Saldo: <saldo> €` e `Transferência efetuada. Saldo: <saldo> €`
Número ou IBAN repetido (`add_account`)	`Já existe uma conta com o número <número>.` e `Já existe uma conta com o IBAN <IBAN>.`
Conta criada	`Conta criada: <conta>`, em que `<conta>` é o `__str__` da conta, e, na linha seguinte, `IBAN: <IBAN>`
Descrição dos movimentos de transferência	`Para <IBAN de destino>` e `De <IBAN de origem>`
Erro num menu	`Erro: <mensagem da exceção>`, com `messages.ERROR_PREFIX`

8. Dados Iniciais

O ficheiro `data.py` é fornecido completo e não pode ser alterado. A função `load_initial_data` cria três contas e o respetivo histórico usando os métodos das classes (`deposit`, `withdraw` e `Bank.transfer`): por isso, só funciona depois de essas classes estarem implementadas. Com os dados carregados, o banco tem as contas e os movimentos indicados na tabela.

Conta	Titular	Tipo	PIN	Saldo	Movimentos
1001	João Silva	Conta à ordem	1234	1130.75 €	11
1002	Maria Costa	Conta poupança	4321	2449.00 €	2
1003	Pedro Santos	Conta à ordem	5678	600.00 €	1

9. Cenários de Teste

A avaliação é feita com os cenários seguintes. Cada cenário começa com o programa acabado de arrancar, com os dados da secção 8. Para os correr todos de uma vez, execute, na pasta que contém a pasta `multibanco`, o comando `python corrigir_cenarios.py multibanco`. O script indica, para cada cenário falhado, a mensagem que faltou. Antes de entregar, todos os cenários devem passar.

Nº	O que fazer	Resultado esperado
T01	Entrar como cliente na conta 1001 com o PIN 1234.	`Bem-vindo(a), João Silva.`
T02	Entrar como cliente na conta 9999.	`Conta não encontrada.`
T03	Conta 1001: introduzir o PIN 0000 e depois o PIN 1234.	`PIN incorreto.` `Bem-vindo(a), João Silva.`

T04	Conta 1001: introduzir três PIN errados seguidos. Voltar a tentar entrar na mesma conta (o PIN já não é pedido).	`PIN incorreto.` (duas vezes)
		`PIN incorreto. A conta foi bloqueada.`
		`Conta bloqueada. Contacte o administrador.`

T05	Conta 1001: consultar o saldo.	`Saldo disponível: 1130.75 €`
T06	Conta 1001: levantar 50.	`Levantamento efetuado. Saldo: 1080.75 €`
T07	Conta 1001: levantar 55.	`Erro: O valor do levantamento tem de ser múltiplo de 10 €.`
T08	Conta 1001: levantar abc.	`Erro: Valor inválido. Introduza um número.`

T09	Conta 1003: levantar 750, depois levantar 60 e consultar o saldo.	`Levantamento efetuado. Saldo: -150.00 €`
		`Erro: Saldo insuficiente.`
		`Saldo disponível: -150.00 €`

T10	Conta 1001: depositar 25,50 (com vírgula).	`Depósito efetuado. Saldo: 1156.25 €`
T11	Conta 1001: depositar 10.555.	`Erro: O valor tem de ser positivo e ter no máximo 2 casas decimais.`

T12	Conta 1002: levantar 100 e consultar os movimentos.	`Levantamento efetuado. Saldo: 2348.00 €`
		`#004 | Comissão | -1.00 € | Comissão de levantamento`
		`#003 | Levantamento | -100.00 € | Levantamento ATM`

T13	Conta 1002: levantar 2450 e consultar o saldo.	`Erro: Saldo insuficiente para o levantamento e a comissão.` `Saldo disponível: 2449.00 €`

T14	Conta 1001: transferir 100 para PT50000000000000000001003. Entrar na conta 1003 e consultar o saldo e os movimentos.	`Transferência efetuada. Saldo: 1030.75 €`
		`Saldo disponível: 700.00 €`
		`#002 | Transferência recebida | +100.00 € | De PT50000000000000000001001`

T15	Conta 1001: transferir 100 para PT50000000000000000009999.	`Erro: IBAN de destino não encontrado.`
T16	Conta 1001: transferir 100 para o seu próprio IBAN.	`Erro: Não é possível transferir para a própria conta.`
T17	Conta 1002: transferir 5000 para a conta 1003. Consultar o saldo das duas contas.	`Erro: Saldo insuficiente.` `Saldo disponível: 2449.00 €` (conta 1002) `Saldo disponível: 600.00 €` (conta 1003)

Nº	O que fazer	Resultado esperado

T18	Conta 1001: consultar os movimentos.	10 linhas, da primeira para a última:
		`#011 | Levantamento | -30.00 € | Levantamento ATM`
		`...`
		`#002 | Levantamento | -50.00 € | Levantamento ATM`

T19	Administrador: criar a conta poupança "Ana Lopes" (PIN 1111, depósito inicial 50). Entrar na conta 1004 e consultar os movimentos.	`Ainda não existem movimentos.`
T20	Entrar como administrador com o PIN 1234 e, de seguida, com o PIN 9999.	`PIN de administrador incorreto.` `Na 2.ª tentativa, aparece o menu de administrador.`

T21	Administrador: listar as contas.	`1001 | Conta à ordem | João Silva | 1130.75 €`
		`1002 | Conta poupança | Maria Costa | 2449.00 €`
		`1003 | Conta à ordem | Pedro Santos | 600.00 €`

T22	Administrador: criar a conta poupança "Ana Lopes" (PIN 1111, depósito inicial 50) e listar as contas.	`Conta criada: 1004 | Conta poupança | Ana Lopes | 50.00 €`
		`IBAN: PT50000000000000000001004`
		A listagem passa a incluir a conta 1004.

T23	Administrador: criar uma conta com o nome "Ana".	`Erro: O nome do titular tem de ter nome e apelido, só com letras.`
T24	Administrador: criar uma conta com o PIN "12a4".	`Erro: O PIN tem de ter 4 dígitos.`
T25	Administrador: criar uma conta escolhendo o tipo 3.	`Erro: Tipo de conta inválido.`
T26	Administrador: eliminar a conta 1001.	`Erro: Só é possível eliminar contas com saldo zero.`
T27	Administrador: eliminar a conta 5555.	`Erro: Conta não encontrada.`
T28	Administrador: criar a conta à ordem "Rui Costa" (PIN 2222, depósito inicial 0) e eliminá-la.	`Conta eliminada.`
T29	Bloquear a conta 1001 com três PIN errados. Entrar como administrador e desbloqueá-la. Voltar a entrar na conta 1001 com o PIN 1234.	`Conta desbloqueada.` `Bem-vindo(a), João Silva.`
T30	Administrador: desbloquear a conta 1002.	`Erro: A conta não está bloqueada.`
T31	Escrever 7 no menu inicial, 9 no menu de cliente e x no menu de administrador.	`Opção inválida.` (nos três menus, sem o programa terminar)
T32	No menu inicial, escolher Sair.	`Obrigado por utilizar o nosso multibanco.` O programa termina com código de saída 0.
T33	Conta 1001: transferir 100 para PT5012345.	`Erro: IBAN inválido.`
T34	Conta 1001: depositar -10.	`Erro: O valor tem de ser positivo e ter no máximo 2 casas decimais.`

Nº	O que fazer	Resultado esperado
T35	Administrador: criar a conta à ordem "Ana Lopes" (PIN 1111) com depósito inicial 10.555.	`Erro: O saldo inicial tem de ser zero ou positivo, com o máximo de 2 casas decimais.`

10. Critérios de Avaliação

O trabalho é avaliado de 0 a 20 valores, com a grelha seguinte. Cada critério é classificado com 1 (cumpre), 0,5 (cumpre parcialmente) ou 0 (não cumpre), e a nota é a soma de cada classificação multiplicada pelo peso do critério.

Bloco	Valores	O que é avaliado
Estrutura e execução	1,5	Estrutura de ficheiros; Arranque do programa; Dados iniciais.
Funções e expressões regulares	2,5	validate_account_number e validate_pin; validate_iban; validate_name; validate_amount; parse_amount.
Classe Movement	1,5	Encapsulamento do movimento; __str__ do movimento; __repr__ do movimento.
Classe Account	4,5	Validação no construtor; Encapsulamento da conta; Verificação do PIN e bloqueio; Depósito; Levantamento; Composição com Movement; Últimos 10 movimentos.
Herança e polimorfismo	3,0	Classes derivadas; Descoberto da conta à ordem; Comissão da conta poupança; Polimorfismo no tipo de conta; __str__ e __repr__ da conta.
Classe Bank e composição	3,0	Contas no banco; Criação de contas; Eliminar e desbloquear; Transferências; Listagem de contas.
Menus e interação	3,0	Menu inicial e acesso de administrador; Autenticação do cliente; Menu de cliente; Menu de administrador; Robustez perante dados inválidos.
Qualidade do código	1,0	Type hints; Docstrings; Convenções do curso.
Total	20	

11. Planeamento Sugerido

No fim de cada aula, o formador faz um ponto de controlo, que não conta para a nota. Serve para confirmar que o trabalho está a avançar ao ritmo necessário.

- Aula 1: leitura conjunta do enunciado e do esqueleto, com demonstração do programa final e do script `corrigir_cenarios.py` (cerca de 30 minutos). Depois, `validators.py`, `Movement` e `Account`, testados com um pequeno script, antes de existirem menus. Ponto de controlo: criar uma conta, depositar, levantar, errar o PIN três vezes e consultar os movimentos.
- Aula 2: `CheckingAccount`, `SavingsAccount` e `Bank`. Ponto de controlo: com `load_initial_data` a correr sem erros, os saldos e o número de movimentos coincidem com a tabela da secção 8.
- Aula 3: `main.py`, com os três menus. Corra `corrigir_cenarios.py`, corrija os cenários que falharem e reveja as regras de escrita do código. Ponto de controlo: o programa completo a funcionar.
- Trabalho autónomo: terminar o que ficou por fazer nas aulas e voltar a correr todos os cenários antes de entregar.